

Name:- Harsh Sharma
Batch:- A2

RollNo:- 16014324028
Subject:- AIR

ExperimentNo:- 4

AIM:- Robot Path Planning: Implement and compare path planning algorithms like A* &Rapidly exploring Random Trees (RRT)

Code:-

```
import numpy as np
import matplotlib.pyplot as plt
import heapq
import random
import time
import math

grid=np.zeros((30,30),dtype=int)
grid[5:25,10]=1
grid[5,10:20]=1
grid[20,10:25]=1
grid[10:25,20]=1
grid[10,20:28]=1
start=(2,2)
goal=(27,27)
def heuristic(a,b):
    return math.sqrt((a[0]-b[0])**2+(a[1]-b[1])**2)
def astar(grid,start,goal):
    open_set=[]
    heapq.heappush(open_set,(0,start))
    came_from={}
    g_score={start:0}
    while open_set:
        _,current=heapq.heappop(open_set)
        if current==goal:
            path=[]
            while current in came_from:
                path.append(current)
                current=came_from[current]
            path.append(start)
            return path[::-1]
```

```

        for dx,dy in
[(1,0),(-1,0),(0,1),(0,-1),(1,1),(1,-1),(-1,1),(-1,-1)]:
            neighbor=(current[0]+dx,current[1]+dy)
            if not(0<=neighbor[0]<grid.shape[0] and
0<=neighbor[1]<grid.shape[1]):
                continue
            if grid[neighbor[0],neighbor[1]]==1:
                continue
            movement=math.sqrt(dx**2+dy**2)
            tentative_g=g_score[current]+movement
            if neighbor not in g_score or tentative_g<g_score[neighbor]:
                came_from[neighbor]=current
                g_score[neighbor]=tentative_g
                f_score=tentative_g+heuristic(neighbor,goal)
                heapq.heappush(open_set,(f_score,neighbor))

    return None

def collision_free(p1,p2,grid):
    distance=heuristic(p1,p2)
    steps=max(int(distance*5),1)
    for i in range(steps+1):
        t=i/steps
        x=round(p1[0]+t*(p2[0]-p1[0]))
        y=round(p1[1]+t*(p2[1]-p1[1]))
        if not(0<=x<grid.shape[0] and 0<=y<grid.shape[1]):
            return False
        if grid[x,y]==1:
            return False
    return True

def
rrt(grid,start,goal,max_iterations=5000,step_size=1.5,goal_threshold=1.5):
    nodes=[start]
    parents={start:None}
    for _ in range(max_iterations):
        if random.random()<0.1:
            sample=goal
        else:
            sample=(random.uniform(0,grid.shape[0]-1),random.uniform(0,grid.shape[1]-1))

        nearest=min(nodes,key=lambda node:heuristic(node,sample))

```

```

        distance=heuristic(nearest,sample)
        if distance==0:
            continue
        ratio=min(step_size/distance,1)

new_node=(nearest[0]+ratio*(sample[0]-nearest[0]),nearest[1]+ratio*(sample
[1]-nearest[1]))
        if not(0<=new_node[0]<grid.shape[0] and
0<=new_node[1]<grid.shape[1]):
            continue
        if not collision_free(nearest,new_node,grid):
            continue
        new_node=(round(new_node[0],3),round(new_node[1],3))
        nodes.append(new_node)
        parents[new_node]=nearest
        if heuristic(new_node,goal)<=goal_threshold and
collision_free(new_node,goal,grid):
            parents[goal]=new_node
            path=[goal]
            current=new_node
            while current is not None:
                path.append(current)
                current=parents[current]
            return path[::-1]
    return None

def path_length(path):
    if path is None:
        return float("inf")
    return sum(heuristic(path[i],path[i+1]) for i in range(len(path)-1))

random.seed(42)
np.random.seed(42)
start_time=time.perf_counter()
astar_path=astar(grid,start,goal)
astar_time=time.perf_counter()-start_time
random.seed(42)
start_time=time.perf_counter()
rrt_path=rrt(grid,start,goal)
rrt_time=time.perf_counter()-start_time
astar_length=path_length(astar_path)
rrt_length=path_length(rrt_path)

```

```

print("A* Path Length:",round(aster_length,3))
print("A* Execution Time:",round(aster_time,6),"seconds")
print("RRT Path Length:",round(rrt_length,3))
print("RRT Execution Time:",round(rrt_time,6),"seconds")
print("A* Path Found:",aster_path is not None)
print("RRT Path Found:",rrt_path is not None)
plt.figure(figsize=(8,8))
plt.imshow(grid.T,origin="lower",cmap="gray_r")
if aster_path:
    ax,ay=zip(*aster_path)
    plt.plot(ax,ay,linewidth=2,label="A*")
if rrt_path:
    rx,ry=zip(*rrt_path)
    plt.plot(rx,ry,linewidth=2,label="RRT")
plt.scatter(start[0],start[1],s=100,marker="o",label="Start")
plt.scatter(goal[0],goal[1],s=100,marker="*",label="Goal")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("A* vs RRT Robot Path Planning")
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(8,8))
plt.imshow(grid.T,origin="lower",cmap="gray_r")
if aster_path:
    ax,ay=zip(*aster_path)
    plt.plot(ax,ay,linewidth=3,label="A* Path")
plt.scatter(start[0],start[1],s=100,marker="o",label="Start")
plt.scatter(goal[0],goal[1],s=100,marker="*",label="Goal")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("A* Path Planning")
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(8,8))
plt.imshow(grid.T,origin="lower",cmap="gray_r")
if rrt_path:
    rx,ry=zip(*rrt_path)
    plt.plot(rx,ry,linewidth=3,label="RRT Path")

```

```

plt.scatter(start[0],start[1],s=100,marker="o",label="Start")
plt.scatter(goal[0],goal[1],s=100,marker="*",label="Goal")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("RRT Path Planning")
plt.legend()
plt.grid(True)
plt.show()
algorithms=["A*", "RRT"]
path_lengths=[astar_length,rrt_length]
execution_times=[astar_time,rrt_time]
plt.figure(figsize=(8,5))
plt.bar(algorithms,path_lengths)
plt.ylabel("Path Length")
plt.title("Path Length Comparison")
plt.show()
plt.figure(figsize=(8,5))
plt.bar(algorithms,execution_times)
plt.ylabel("Execution Time (seconds)")
plt.title("Execution Time Comparison")
plt.show()

```

Output:-

```

A* Path Length: 44.142
A* Execution Time: 0.004506 seconds
RRT Path Length: 52.347
RRT Execution Time: 0.003114 seconds
A* Path Found: True
RRT Path Found: True

```

Graphs & Results:-

Figure 1

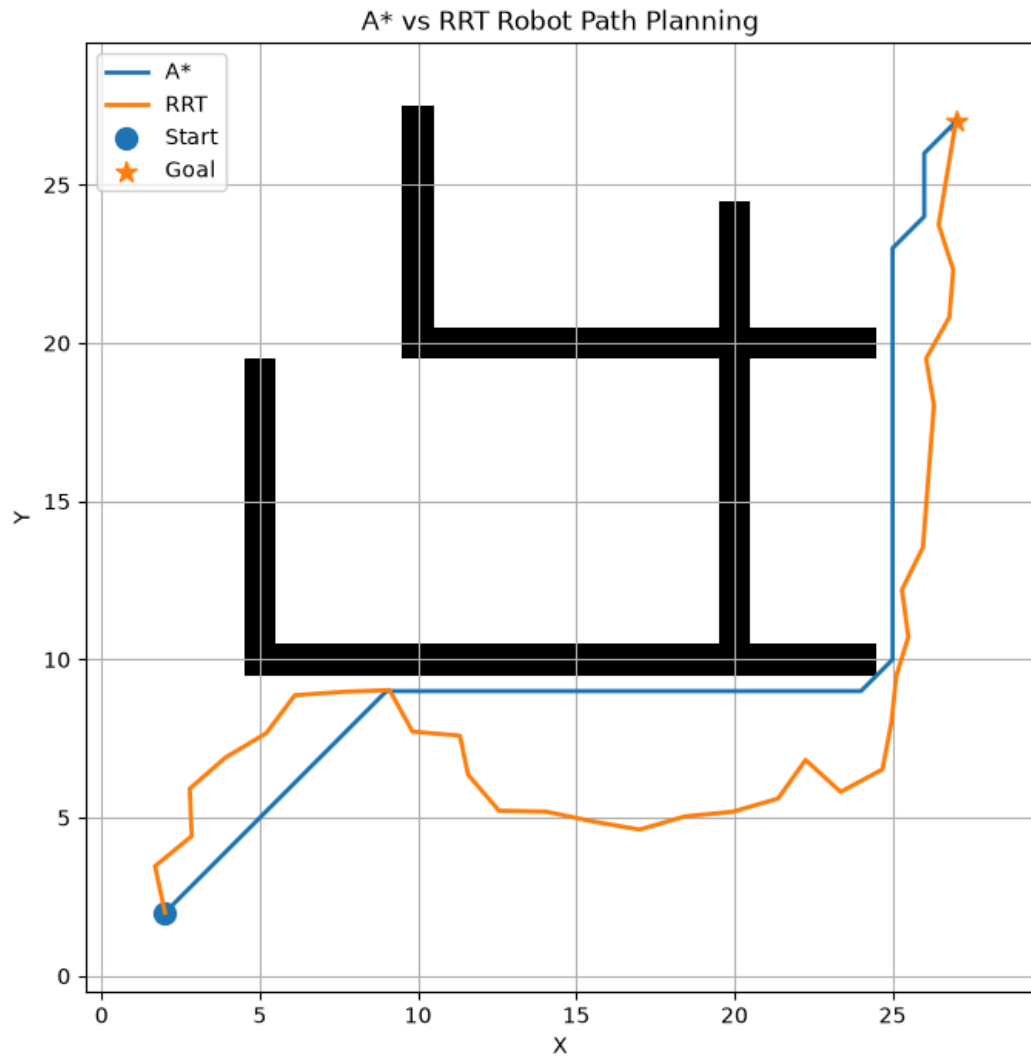


Figure 1

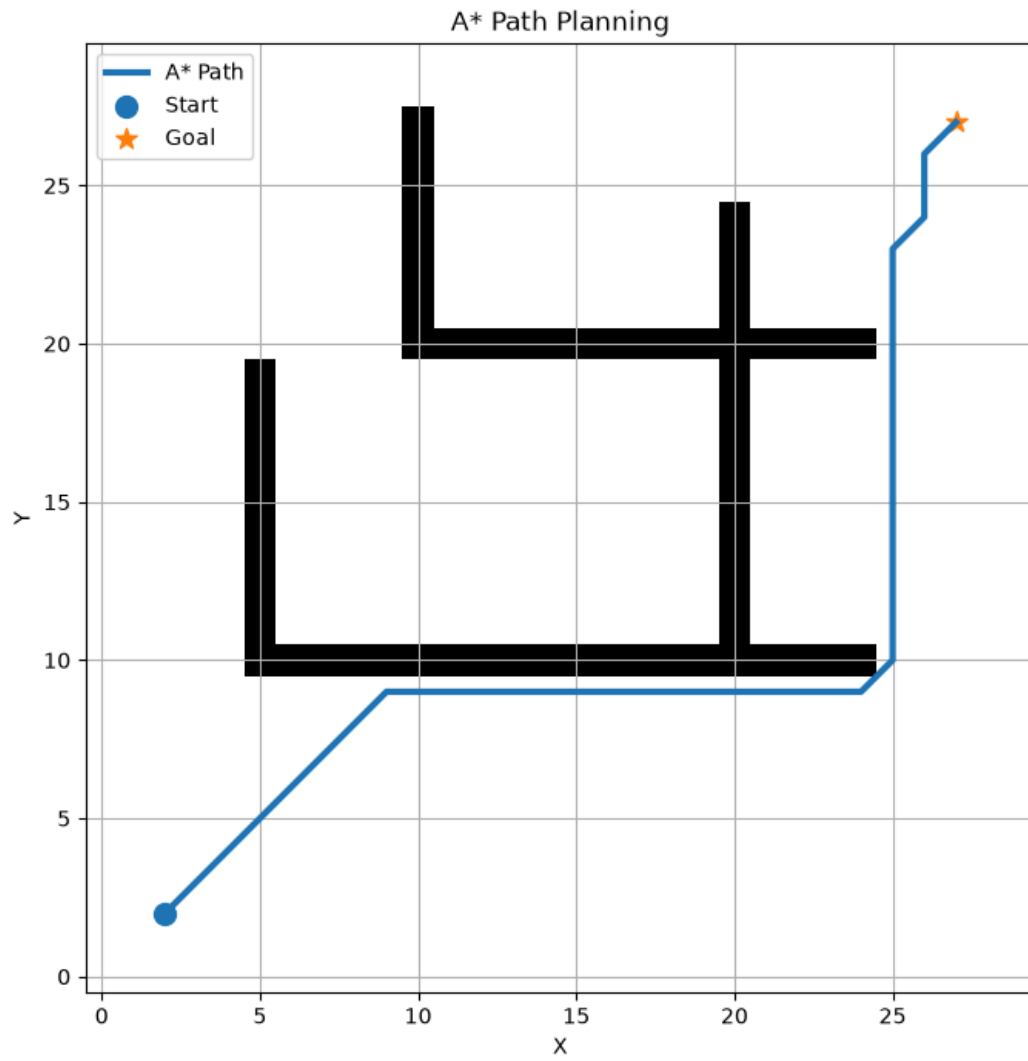


Figure 1

